TALLER DE PROGRAMACIÓN / CLASE #01

Guía Teórica - Docentes

1. Introducción a la programación
2. Presentación de la herramienta: Python
3. Datos fundamentales del lenguaje
4. print vs return
5. Estructura de un programa
6. Variables
7. Métodos / funciones principales e iniciales
8. Indagación del lenguaje: help(), dir(), documentación
9. Tipos de datos y uso de type()
10. Introducción a las listas
11. Operaciones con Strings
12. La función input
99. Ejercitación propuesta

1. Introducción a la programación

Programar es el proceso de escribir instrucciones, en un lenguaje que la computadora pueda interpretar, para que esta ejecute una secuencia de pasos y resuelva un problema o realice una tarea… entre otras: 'programar es comunicar', etc.

Conceptos clave para presentar:

- Algoritmo: secuencia finita y ordenada de pasos que resuelve un problema. Todo programa es la implementación de uno o más algoritmos en un lenguaje concreto.
- Lenguaje de programación: conjunto de reglas sintácticas y semánticas que permiten escribir instrucciones ejecutables por una computadora.
- Código fuente: texto escrito por el programador, en un lenguaje de programación.
- Intérprete vs. compilador: un compilador traduce todo el código fuente a código máquina antes de ejecutarlo; un intérprete lo traduce y ejecuta línea por línea (o expresión por expresión) en tiempo real. Python es un lenguaje interpretado.

Ejemplo conceptual (algoritmo en lenguaje natural):

```
1. Pedir dos números al usuario.
2. Sumarlos.
3. Mostrar el resultado.
```

Esto luego se traduce a código Python:

```
a = int(input("Ingrese el primer número: "))
b = int(input("Ingrese el segundo número: "))
print(a + b)
```

2. Presentación de la herramienta: Python

Python es un lenguaje de programación de alto nivel, interpretado, de tipado dinámico y multiparadigma (soporta programación estructurada, orientada a objetos y funcional), creado por Guido van Rossum y publicado por primera vez en 1991.

Características a destacar:

- Sintaxis legible: usa indentación (espacios/tabs) en lugar de llaves {} para delimitar bloques de código.
- Tipado dinámico: no es necesario declarar el tipo de una variable; se infiere en tiempo de ejecución.
- Multiplataforma: el mismo código corre en Windows, Linux o macOS.
- Gran ecosistema: enorme cantidad de librerías (NumPy, Pandas, Django, etc.).

Entornos de trabajo habituales para un curso inicial:

Entorno	Descripción
Intérprete interactivo (REPL)	Se ejecuta línea por línea desde la terminal escribiendo python
IDLE	Editor incluido con la instalación oficial de Python
Jupyter Notebook / Google Colab	Entorno de celdas, ideal para clases y experimentación
VS Code / PyCharm	Editores/IDEs completos para proyectos más grandes

Ejemplo – primer programa (tradicional "Hola, mundo"):

```
print("Hola, mundo")
```

3. Datos fundamentales del lenguaje

Los datos son la información con la que un programa trabaja. En Python, cada dato en memoria es un objeto, y todo objeto tiene un tipo asociado.

Elementos fundamentales a introducir:

- Literal: representación fija de un valor en el código (5, "hola", True).
- Expresión: combinación de valores, variables y operadores que produce un resultado (3 + 4).
- Operadores:
  Aritméticos: + - * / // % **
  Comparación: == != > < >= <=
  Lógicos: and or not
  Asignación: = += -= *= /=

Ejemplo:

```
5 + 3          # expresión aritmética -> 8
5 == 3         # expresión de comparación -> False
10 // 3        # división entera -> 3
10 % 3         # resto de la división -> 1
2 ** 4         # potencia -> 16
```

4. print vs return

Este es uno de los puntos que más confusión genera en estudiantes principiantes, conviene remarcar.

print() - Es una función que muestra información por pantalla (salida estándar). No devuelve un valor útil: técnicamente devuelve None. Sirve para comunicar algo al usuario, no para que otra parte del programa use ese valor.

return - Es una sentencia que se usa dentro de una función para devolver un valor a quien la llamó. Ese valor puede guardarse en una variable, usarse en un cálculo, pasarse a otra función, etc. Corta la ejecución de la función en el momento en que se ejecuta.

Ejemplo comparativo:

```
def sumar_print(a, b):
    print(a + b)      # muestra el resultado, pero no lo "entrega"

def sumar_return(a, b):
    return a + b      # entrega el resultado para ser usado

resultado1 = sumar_print(5, 3)   # en pantalla se ve "8"
print(resultado1)                # pero resultado1 es None

resultado2 = sumar_return(5, 3)  # no se ve nada en pantalla
print(resultado2)                # ahora sí: 8
```

Idea clave para transmitir: print es para mostrar, return es para devolver y reutilizar. Una función puede tener return y jamás usar print, y viceversa.

5. Estructura de un programa

Un programa en Python es un conjunto ordenado de sentencias que se ejecutan secuencialmente de arriba hacia abajo, salvo que estructuras de control (condicionales, bucles, funciones) alteren ese flujo.

Elementos típicos que componen un programa:

- Comentarios: texto ignorado por el intérprete, usado para documentar (# esto es un comentario).
- Importaciones: incorporación de módulos externos (import math).

- Declaración de variables y constantes.
- Definición de funciones.
- Bloque principal: la secuencia de instrucciones que efectivamente se ejecuta.

Ejemplo de programa simple y completo:

```
# Programa que calcula el área de un círculo
import math

def calcular_area(radio):
    return math.pi * radio ** 2

radio = float(input("Ingrese el radio: "))
area = calcular_area(radio)
print("El área es:", area)
```

¡Importante! La función input() sirve para pedirle datos al usuario desde el teclado durante la ejecución del programa.

- Cuando Python llega a una línea con input(), pausa la ejecución y espera que el usuario escriba algo y presione Enter.
- Opcionalmente podés pasarle un texto como argumento, que se muestra como mensaje/prompt antes de esperar la entrada.
- Lo que el usuario escribe queda siempre como un string (str), sin importar si escribió números o letras.

6. Variables

Una variable es un nombre simbólico asociado a un valor almacenado en memoria. En Python, una variable es en realidad una referencia (etiqueta) que apunta a un objeto en memoria; no es una "caja" que contiene el valor directamente, sino un nombre que "señala" hacia él.

Algunas convenciones (¡no es sintaxis!)

- Deben comenzar con una letra o guión bajo (_), nunca con un número.
- Pueden contener letras, números y guiones bajos.
- Son case-sensitive (edad y Edad son variables distintas).
- No se deben usar palabras reservadas del lenguaje (if, for, class, etc.).
- Convención de escritura: snake_case (nombre_completo, edad_usuario).

Asignación:

```
edad = 20               # asignación simple
nombre = "Ana"          # reasignable en cualquier momento
edad = edad + 1         # reasignación usando el valor anterior
x, y, z = 1, 2, 3       # asignación múltiple
a = b = 0               # misma referencia para varias variables
```

Idea clave: en Python las variables no tienen tipo fijo; el tipo lo tiene el objeto al que la variable apunta en cada momento.

```
x = 5             # x referencia un entero
x = "cinco"       # ahora x referencia un string; es totalmente válido
```

7. Métodos / funciones principales e iniciales

Conviene distinguir dos conceptos desde el principio:

- Función: bloque de código reutilizable e independiente, se invoca por su nombre (print(), len(), type()).
- Método: función que "pertenece" a un objeto y se invoca con la notación punto (texto.upper(), lista.append()).

Funciones incorporadas (built-in) más usadas en un curso inicial:

Función	Descripción	Ejemplo
print()	Muestra en pantalla	print("Hola")
input()	Lee texto ingresado por teclado	input("Nombre: ")
int(), float(), str(), bool()	Conversión entre tipos	int("5")
len()	Longitud de una secuencia	len("hola")
type()	Tipo de un objeto	type(5)
range()	Genera una secuencia de números	range(0, 10)
sum(), max(), min()	Operaciones sobre colecciones	max([1, 5, 2])
abs()	Valor absoluto	abs(-3)
round()	Redondeo	round(3.14159, 2)

Ejemplo:

```
nombre = input("¿Cómo te llamás? ")
print("Hola,", nombre, "- tu nombre tiene", len(nombre), "letras")
```

8. Indagación del lenguaje

Python ofrece herramientas propias del lenguaje para explorar en tiempo real qué hace un objeto, una función o un módulo, sin necesidad de salir del entorno de trabajo. Es importante enseñar esto temprano para fomentar la autonomía del estudiante.

Herramientas principales:

- help(objeto): muestra la documentación (docstring) asociada a una función, método, módulo o tipo.
- dir(objeto): lista todos los atributos y métodos disponibles de un objeto.
- type(objeto): informa el tipo del objeto (ver sección siguiente).
- id(objeto): muestra la dirección de memoria (identidad) del objeto.
- Docstrings ("""texto"""): forma estándar de documentar funciones propias, accesible luego con help().

Ejemplo:

```
help(str.upper)     # muestra qué hace el método upper() de los strings
dir("hola")         # lista todos los métodos disponibles para strings

def saludar(nombre):
    """Devuelve un saludo personalizado para 'nombre'."""
    return f"Hola, {nombre}"

help(saludar)       # muestra el docstring definido arriba
```

9. Tipos de datos y uso de type()

Un tipo de dato define qué valores puede tomar un objeto y qué operaciones son válidas sobre él. En Python, todo valor es un objeto y tiene un tipo, que puede consultarse dinámicamente con la función type().

Tipos de datos básicos (built-in):

Tipo	Nombre en Python	Ejemplo	Mutable
Entero	int	5, -3, 1000	No
Flotante	float	3.14, -0.5	No
Booleano	bool	True, False	No
Cadena de texto	str	"hola", “@$#”	No
Lista	list	[1, 2, 3]	Sí
Tupla	tuple	(1, 2, 3)	No
Diccionario	dict	{"clave": "valor"}	Sí
Conjunto	set	{1, 2, 3}	Sí
Nulo	NoneType	None	—

Uso de type():

```
type(5)          # <class 'int'>
```

```
type(3.14)        # <class 'float'>
type("hola")      # <class 'str'>
type(True)        # <class 'bool'>
type([1, 2, 3])   # <class 'list'>
type(None)        # <class 'NoneType'>
```

Algunos lenguajes (como JavaScript) usan el operador typeof para esto. En Python el equivalente es la función type(), no un operador.

Conversión de tipos (type casting):

```
int("10")         # convierte string a entero -> 10
float("3.5")      # convierte string a flotante -> 3.5
str(25)           # convierte entero a string -> "25"
bool(0)           # 0 se convierte en False
bool(1)           # cualquier valor distinto de 0 se convierte en True
```

Verificación de tipo con isinstance() (complemento útil a type()):

```
isinstance(5, int)                  # True
isinstance("hola", (int, float))    # False
```

10. Introducción a las listas

Una lista (list) es una colección ordenada y mutable de elementos, que pueden ser de distintos tipos, incluyendo otras listas. Se define con corchetes [ ].

Características principales:

- Ordenada: cada elemento tiene una posición (índice), comenzando en 0.
- Mutable: se puede modificar después de creada (agregar, eliminar o cambiar elementos).
- Heterogénea: puede mezclar tipos de datos distintos.
- Indexable y "recortable" mediante slicing.

Creación y acceso:

```
frutas = ["manzana", "banana", "pera"]
frutas[0]        # "manzana" (primer elemento, índice 0)
frutas[-1]       # "pera" (último elemento, índice negativo)
frutas[1:3]      # ["banana", "pera"] (slicing)
```

Métodos principales de listas:

Método	Descripción	Ejemplo
append(x)	Agrega un elemento al final	frutas.append("uva")

insert(i, x)	Inserta en una posición específica	frutas.insert(0, "kiwi")
remove(x)	Elimina la primera coincidencia	frutas.remove("pera")
pop(i)	Elimina y devuelve el elemento en índice i	frutas.pop(0)
sort()	Ordena la lista in situ	frutas.sort()
reverse()	Invierte el orden	frutas.reverse()
len()	Longitud de la lista (función, no método)	len(frutas)

Ejemplo integrador:

```
numeros = [4, 2, 9, 1]
numeros.append(7)       # [4, 2, 9, 1, 7]
numeros.sort()          # [1, 2, 4, 7, 9]
print("El mayor es:", numeros[-1])
print("Cantidad de elementos:", len(numeros))
```

11. Operaciones con Strings

Un string (str) es una secuencia inmutable de caracteres. "Inmutable" significa que, una vez creado, no puede modificarse en el lugar: cualquier operación que "cambia" un string en realidad crea uno nuevo.

Creación:

```
texto = "Hola, mundo"
texto2 = 'También válido con comillas simples'
texto3 = """Puede ser
multilínea"""
```

Indexación y slicing (igual que en listas, porque un string es una secuencia):

```
texto[0]      # "H"
texto[-1]     # "o"
texto[0:4]    # "Hola"
```

Concatenación y repetición:

```
saludo = "Hola" + " " + "mundo"     # concatenación -> "Hola mundo"
risa = "ja" * 3                     # repetición -> "jajaja"
```

Métodos principales de strings:

Método	Descripción	Ejemplo	Resultado
.upper()	Convierte a mayúsculas	"hola".upper()	"HOLA"
.lower()	Convierte a minúsculas	"HOLA".lower()	"hola"
.strip()	Elimina espacios al inicio y/o al fin	" hola ".strip()	"hola"
.replace(a, b)	Reemplaza subcadenas	"hola".replace("h", "j")	"jola"
.split(sep)	Divide en una lista	"a,b,c".split(",")	["a", "b", "c"]
.join(lista)	Une elementos de una lista	"-".join(["a","b"])	"a-b"
.find(x)	Índice de la primera aparición	"hola".find("l")	2
.count(x)	Cantidad de apariciones	"banana".count("a")	3
.startswith(x) / .endswith(x)	Verifica inicio/fin	"hola".startswith("ho")	True

Formateo de strings (muy usado en la práctica):

```
nombre = "Ana"
edad = 20

# f-strings (forma moderna y recomendada)
print(f"{nombre} tiene {edad} años")

# método .format()
print("{} tiene {} años".format(nombre, edad))
```

Ejemplo integrador:

```
frase = "  La Programación es divertida  "
frase_limpia = frase.strip().lower()
palabras = frase_limpia.split()
print(f"La frase tiene {len(palabras)} palabras")
print("Palabras:", palabras)
```

Cierre pedagógico sugerido

Para consolidar estos temas en una primera unidad, resulta útil que el estudiante practique:

- Escribir programas breves que combinen input(), variables y print().
- Diferenciar explícitamente casos de uso de print vs return mediante funciones propias.
- Explorar tipos con type() sobre distintos valores y conversiones.
- Manipular listas de strings, aplicando métodos de ambas estructuras en conjunto (por ejemplo, split() seguido de operaciones de lista).

Esta base conceptual permite avanzar luego hacia estructuras de control (if, for, while), funciones más complejas y estructuras de datos adicionales (tuplas, diccionarios, conjuntos).

99. Ejercitación propuesta

EJ01) Creá tres variables: nombre, apellido y edad. Luego mostrará un mensaje que las combine en una sola oración.

EJ02) Dados a = 17 y b = 5, calcula e imprime: la suma, la resta, el producto, la división real, la división entera y el resto.

EJ03) Definir dos funciones que calculen el área de un rectángulo: una que la muestre con print (area_print) y otra que la devuelva con return (area_return). Después, intentá guardar el resultado de cada una en una variable e imprimirlo, para comprobar la diferencia.

EJ04) Escribí un programa que le pida al usuario dos números, calcule su promedio usando una función con return, y muestre el resultado con print.

EJ05) Pedile al usuario que ingrese una dirección de correo electrónico. Mostrá en pantalla su longitud, una versión donde el símbolo "@" sea reemplazado por la palabra "[at]", y el resultado lógico (`True` o `False`) de evaluar si el correo termina con ".com".

EJ06) Usá type(), dir() y help() sobre un mismo string para investigar qué operaciones admite, y probá al menos dos métodos que encuentres en dir() que no hayan sido vistos en clase.

EJ07) Escribí un programa que le pida al usuario un número decimal (idealmente negativo, como -4.567). Calculá y mostrá su valor absoluto. Luego redondealo a un decimal, y por último convertilo a número entero. Imprimí el valor final y verificá su tipo de dato final en pantalla.

EJ08) Creá una lista con 5 nombres de frutas. Agregá una nueva fruta al final, eliminá la primera, ordena la lista alfabéticamente y mostrá cuántos elementos quedaron.

EJ09) Pedile al usuario que ingrese una frase de al menos tres palabras. Limpia los espacios sobrantes a los extremos, convertila a mayúsculas, y separala en una lista de palabras. Mostrá la cantidad total de palabras. Luego, extrae (usando *slicing*) solo las primeras dos palabras de esa lista, y unilas nuevamente en un único string separadas por un guión medio ("-").

EJ10) Definí una función que reciba tres palabras por parámetro. La función debe crear una lista con ellas, ordenarla alfabéticamente (in situ) y devolver (con return) la primera palabra de la lista ya ordenada. En el bloque principal del programa, pedile al usuario tres palabras distintas, llamá a tu función pasándole esos argumentos, y mostrá el resultado devuelto en pantalla.